Facade Design Pattern

Study notes - structural pattern, first-principles walkthrough with FAANG interview angle

1. The Problem (First Principles)

Real-life analogy: at a restaurant you do not walk into the kitchen, talk to the butcher, the sommelier, and the chef yourself, and time everything perfectly. You tell a **waiter** "I'll have the salmon" and the waiter coordinates the whole subsystem behind the scenes.

In software, a **subsystem** is often made of many interdependent classes, each with its own interface and its own rules about ordering and state. Turning on a home theater, for example, means correctly sequencing: dim the lights, lower the screen, power the projector, set it to widescreen, power the amplifier, set the volume, power the DVD player, press play - every single time.

If every client (every screen, every script) has to know that exact sequence, you get duplicated, fragile, tightly coupled code everywhere that sequence is needed. Change the subsystem's internal ordering once and you must hunt down every call site.

That is the problem Facade solves: **too many moving parts, no single simple entry point.**

2. Standard (GoF) Definition

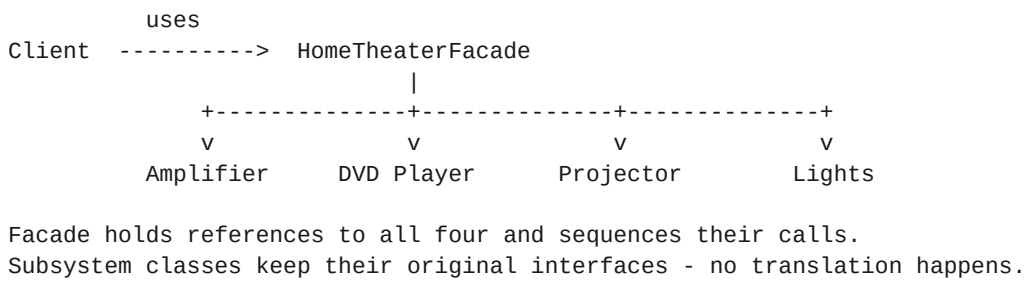
"Provide a unified interface to a set of interfaces in a subsystem. Facade defines a higher-level interface that makes the subsystem easier to use."

It is a **structural** pattern, like Adapter - but where Adapter translates one interface into another (one-to-one), Facade simplifies access to many classes at once (many-to-one).

3. Structure & Participants

Two roles, one fan-out - notice there is no dashed "implements" arrow like in Adapter:

- **Client** - your code (e.g. a "Watch Movie" button handler). Talks to exactly one object: the Facade.
- **Facade** (HomeTheaterFacade) - holds references to every subsystem class it needs, exposes simple methods like watchMovie() and endMovie().
- **Subsystem classes** (Amplifier, DVD Player, Projector, Lights) - unchanged, original interfaces still directly usable. They do not know the Facade exists.



Text rendering of the UML structure: Client -> Facade (association), Facade -> each subsystem class (composition/has-a, no realization arrow - interfaces are not translated, just coordinated).

4. Code Example

The problem - without a Facade

```
Amplifier amp = new Amplifier();
DvdPlayer dvd = new DvdPlayer();
Projector projector = new Projector();
TheaterLights lights = new TheaterLights();
Screen screen = new Screen();

// Every single time you want to watch a movie, somewhere in your codebase:
lights.dim(10);
screen.down();
projector.on();
projector.wideScreenMode();
amp.on();
amp.setVolume(5);
dvd.on();
dvd.play("Inception");
```

If this exact sequence is needed in three different screens of your app, you now have three copies of fragile, order-dependent code. Forget screen.down() before projector.on() in just one of them and you have a subtle bug that is painful to track down.

The solution - with a Facade

```
class HomeTheaterFacade {
    private Amplifier amp;
    private DvdPlayer dvd;
    private Projector projector;
    private TheaterLights lights;
    private Screen screen;

    public HomeTheaterFacade(Amplifier amp, DvdPlayer dvd, Projector projector,
                             TheaterLights lights, Screen screen) {
        this.amp = amp; this.dvd = dvd; this.projector = projector;
        this.lights = lights; this.screen = screen;
    }

    public void watchMovie(String movie) {
        lights.dim(10);
        screen.down();
        projector.on();
        projector.wideScreenMode();
        amp.on();
        amp.setVolume(5);
        dvd.on();
        dvd.play(movie);
    }

    public void endMovie() {
        lights.dim(100);
        screen.up();
        projector.off();
        amp.off();
        dvd.stop();
        dvd.off();
    }
```

```
}

// Usage
HomeTheaterFacade homeTheater = new HomeTheaterFacade(amp, dvd, projector, lights, screen);
homeTheater.watchMovie("Inception");   // one call, correct sequence guaranteed
homeTheater.endMovie();
```

The subsystem classes were not touched at all. Amplifier, DvdPlayer, etc. keep their full original interfaces - the Facade just gives you a shortcut. Advanced users can still call amp.setVolume(11) directly. This is different from Adapter, where the client is forced through the translated interface.

5. Types / Variants of Facade

Type 1 - Basic (Simple) Facade

One facade class, one subsystem, straightforward simplification - the home theater example above.

Analogy: a single TV remote's "Movie Mode" button - one press triggers dimmed lights, closed blinds, and the right input source.

Type 2 - Layered Facade (Facade of Facades)

In larger systems you build facades over facades. A big subsystem might have its own InventoryFacade, PaymentFacade, and ShippingFacade, and a top-level OrderFacade composes those three into one placeOrder() call.

Analogy: a hotel's General Manager does not personally make every bed or cook every dish. They delegate to a Housekeeping Manager and an Executive Chef, each already a simplified point of contact for their own department. The GM is a facade over facades.

```
class OrderFacade {
    private InventoryFacade inventory;
    private PaymentFacade payment;
    private ShippingFacade shipping;

    public void placeOrder(Order order) {
        inventory.reserveItems(order);
        payment.charge(order);
        shipping.schedule(order);
    }
}
```

Type 3 - Remote Facade

When subsystem calls cross a network boundary, every fine-grained call is expensive (latency, serialization). A Remote Facade batches several calls into one coarse-grained network call.

Analogy: booking an overseas trip through one call to a travel agent, instead of separately calling the airline, the hotel, and the car rental company yourself across expensive long-distance lines.

This is directly why the **API Gateway pattern** in microservices exists - a Remote Facade at the network layer, letting a mobile client make one request instead of five.

Variant	What it hides	Real-life analogy
Basic Facade	A handful of classes in one subsystem	TV remote “Movie Mode” button
Layered Facade	Multiple facades, each over their own subsystem	Hotel GM delegating to department managers
Remote Facade	Multiple expensive network calls	One call to a travel agent instead of five to vendors

6. Real-World Use Cases

- **SLF4J:** a facade over multiple logging frameworks (Log4j, java.util.logging, Logback) - one simple logging interface, SLF4J picks which framework does the work.
- **jQuery** (historically): a facade over verbose, inconsistent, cross-browser DOM APIs.
- **Compiler front-ends:** a single compile() call hides the lexer, parser, semantic analyzer, and code generator working in sequence.
- **API Gateway** in microservices: a facade over dozens of backend services, one simplified endpoint for clients.
- **ORMs** (Hibernate, ActiveRecord): save() hides connection management, SQL generation, transaction handling, and dirty-checking.
- **Cloud SDK clients:** e.g. a single S3Client object that hides authentication, retries, request signing, and low-level HTTP calls.

7. FAANG Interview Angle - Facade vs Its Cousins

- **vs Adapter:** Adapter is one-to-one - it translates one incompatible interface into another the client already expects, and the client is forced through it to get the behavior at all. Facade is many-to-one - it simplifies access to a subsystem whose interfaces were never incompatible, just numerous. With Facade the subsystem's original interfaces usually remain directly accessible too.
- **vs Mediator:** Facade is one-way - client calls facade, facade calls subsystem classes; subsystem classes do not call each other through the facade. Mediator centralizes peer-to-peer communication - the objects do need to talk to each other, and the Mediator is the hub coordinating that.
- **vs Proxy:** Proxy implements the same interface as the real object it stands in for (lazy loading, access control, remote calls) - the client cannot tell the difference. Facade deliberately creates a brand-new, simplified interface that looks nothing like the subsystem's original interfaces.
- **Common curveball:** “Does adding a new subsystem class force you to modify the Facade - doesn't that violate OCP?” Good answer: yes, somewhat - the Facade is the one place intentionally coupled to the subsystem's internals, isolating that cost to a single class instead of spreading it across every client.
- **Another common question:** “Can you have multiple facades over the same subsystem?” Yes - e.g. an AdminFacade exposing dangerous operations and a UserFacade exposing only safe, everyday operations, both sitting over the same underlying classes.

8. Memory Map (Quick Recall)

```
FACADE PATTERN
|
+-- PROBLEM: Client must correctly sequence calls across many interdependent
|     subsystem classes just to do one simple thing
```

```
|     (analogy: ordering through a waiter instead of running the kitchen yourself)
|
+-- DEFINITION: Provide a unified, higher-level interface to a set of interfaces
|               in a subsystem, making the subsystem easier to use
|
+-- 2 PARTICIPANTS (no dashed "implements" arrow, unlike Adapter)
|     +-- Client   -> talks only to the Facade
|     +-- Facade   -> holds references to subsystem classes, sequences their calls
|     +-- Subsystem classes -> unchanged, original interfaces still directly usable
|
+-- 3 VARIANTS
|     +-- Basic Facade    -> one facade, one subsystem
|     +-- Layered Facade  -> facade of facades (facade composing other facades)
|     +-- Remote Facade   -> batches expensive network calls (-> API Gateway)
|
+-- REAL EXAMPLES: SLF4J, jQuery, compiler front-ends, API Gateway, ORMs
|
+-- INTERVIEW DIFFERENTIATORS
      +-- vs Adapter   -> many:1 simplification, not 1:1 interface translation
      +-- vs Mediator  -> one-way (client->subsystem), not peer-to-peer coordination
      +-- vs Proxy     -> new simplified interface, not same interface as real object
      +-- OCP tradeoff -> new subsystem class = facade must change (deliberate, contained)
```

One sentence to remember: Facade doesn't change any interfaces - it just gives the client one simple door into a room full of complicated furniture.